

UNIVERSIDADE FEDERAL DO PARÁ

FACULDADE DE ENGENHARIA ELÉTRICA E BIOMÉDICA

SISTEMAS DE CONTROLE I

MARCOS VINÍCIUS RIBEIRO CORRÊA

MODELAGEM DE SISTEMAS DE PRIMEIRA E SEGUNDA ORDEM EM C:

ANÁLISE DAS CARACTERÍSTICAS DA RESPOSTA AO DEGRAU

BELÉM

2026

1 OBJETIVO

O objetivo deste trabalho é identificar as Funções de Transferência G(s) de sistemas lineares invariantes no tempo (LTI) a partir de parâmetros extraídos graficamente da resposta temporal ao degrau de amplitude A. Para isso, foi desenvolvido um programa em linguagem C que calcula os parâmetros dos modelos e exporta a resposta simulada em arquivos CSV, para plotagem em planilha eletrônica. O código completo encontra-se no Apêndice A.

São tratados dois tipos de sistema. O sistema de 1ª ordem não apresenta sobressinal: sua resposta cresce de forma monotônica e exponencial até o valor de regime e é caracterizada pela constante de tempo T. O sistema de 2ª ordem subamortecido ($0 < \zeta < 1$) possui polos complexos conjugados e, por isso, apresenta sobressinal e oscilação amortecida antes de se estabilizar; ele é caracterizado pelo fator de amortecimento ζ e pela frequência natural ω_n.

2 PROCESSO DE IDENTIFICAÇÃO DE VARIÁVEIS

Os parâmetros dos modelos são calculados a partir de quatro grandezas lidas na curva gráfica da resposta ao degrau. A amplitude da entrada e o valor de estabilização são comuns aos dois tipos de sistema; o que muda é o ponto da curva usado para determinar a dinâmica.

2.1 TIPO A: SISTEMA DE 1ª ORDEM

Para o sistema de 1ª ordem, devem ser extraídas as seguintes variáveis:

a) amplitude da entrada, A (variável `amplitude`);
b) valor de estabilização em regime permanente, $y(\infty)$ (variável `estabilizador`);
c) instante de tempo de amostragem, t_1 (variável `tempo`);
d) valor da resposta no instante t_1, $y(t_1)$ (variável `ytempo`).

2.2 TIPO B: SISTEMA DE 2ª ORDEM SUBAMORTECIDO

Para o sistema de 2ª ordem subamortecido, devem ser extraídas as seguintes variáveis:

a) amplitude da entrada, A (variável `amplitude`);
b) valor de estabilização em regime permanente, $y(\infty)$ (variável `estabilizador`);
c) valor máximo do primeiro pico, y_p (variável `yp`);
d) tempo em que ocorre o primeiro pico, t_p (variável `tp`).

3 DEDUÇÕES E MANIPULAÇÕES MATEMÁTICAS

Esta seção apresenta, passo a passo, as equações que relacionam as variáveis lidas no gráfico aos parâmetros fundamentais de cada sistema.

3.1 MODELAGEM DE 1ª ORDEM (TIPO A)

O ganho estático (ganho DC) é a razão entre o valor de regime e a amplitude do degrau, conforme a Equação 1. A resposta ao degrau é dada pela Equação 2.

$$K_{\mathrm{DC}} = \frac{y(\infty)}{A} \tag{1}$$

$$y(t) = y(\infty)\left(1 - e^{-t/T}\right) \tag{2}$$

Impondo que a curva passe pelo ponto $\left(t_1, y(t_1)\right)$, tem-se $\frac{y(t_1)}{y(\infty)} = 1 - e^{-\frac{t_1}{T}}$. Aplicando o logaritmo natural e isolando T, obtém-se a Equação 3. Com ela, a função de transferência é montada pela Equação 4.

$$T = \frac{-t_1}{\ln\left(1 - \frac{y(t_1)}{y(\infty)}\right)} \tag{3}$$

$$G_1(s) = \frac{K_{\mathrm{DC}}}{T \cdot s + 1} \tag{4}$$

3.2 MODELAGEM DE 2ª ORDEM SUBAMORTECIDA (TIPO B)

O ganho DC é calculado como na Equação 1. O sobressinal relativo (overshoot) é dado pela Equação 5.

$$M_p = \frac{y_p - y(\infty)}{y(\infty)} \tag{5}$$

Para um sistema subamortecido, o sobressinal se relaciona ao fator de amortecimento pela Equação 6. Aplicando o logaritmo natural, elevando ao quadrado e isolando ζ, obtém-se a expressão exata da Equação 7.

$$M_p = e^{-\frac{\pi\zeta}{\sqrt{1-\zeta^2}}} \tag{6}$$

$$\zeta = \frac{-\ln(M_p)}{\sqrt{\pi^2 + \ln(M_p)^2}} \tag{7}$$

O primeiro pico ocorre em $t_p = \frac{\pi}{\omega_d}$, com a frequência amortecida $\omega_d = \omega_n\sqrt{1 - \zeta^2}$. Logo, a frequência natural é dada pela Equação 8, e a função de transferência, pela Equação 9.

$$\omega_n = \frac{\pi}{t_p\sqrt{1-\zeta^2}} \tag{8}$$

$$G_2(s) = \frac{K_{DC}{\omega_n}^2}{s^2 + 2\zeta\omega_n s + {\omega_n}^2} \quad (9)$$

A resposta temporal subamortecida, usada na simulação, é dada pela Equação 10, com $\varphi = arccos(\zeta)$.

$$y_2(t) = y(\infty)\left(1 - \frac{e^{-\zeta\omega_n t}}{\sqrt{1-\zeta^2}}\text{sen}(\omega_d t + \varphi)\right) \quad (10)$$

4 IMPLEMENTAÇÃO COMPUTACIONAL E FORMATAÇÃO DE DADOS PARA EXPORTAÇÃO

O programa foi estruturado de forma modular, com uma função para cada tipo de sistema e uma função principal que escolhe entre elas. As subseções a seguir explicam cada parte.

4.1 FUNÇÃO PRINCIPAL

```
scanf(" %c", &tipo);
if(tipo == 'A' || tipo == 'a'){ tipoA(); }
else if(tipo == 'B' || tipo == 'b'){ tipoB(); }
```

A função `main()` lê um caractere com o tipo de sistema e chama `tipoA()` ou `tipoB()`, aceitando maiúsculas e minúsculas. O espaço antes de `%c` faz o `scanf` ignorar a quebra de linha pendente. Para qualquer outra entrada, o programa exibe a mensagem "Tipo invalido.".

4.2 FUNÇÃO TIPOA()

```
float Kdc = estabilizador/amplitude;
double T = ((-1)*tempo)/log(1-(ytempo/estabilizador));
```

A função lê as quatro variáveis do Tipo A com `scanf("%f")`. A primeira linha acima implementa a Equação 1 e a segunda, a Equação 3, na qual `log` é o logaritmo natural da biblioteca `math.h`. Em seguida, o programa imprime Kdc, T e a função de transferência G(s) = Kdc/(T·s + 1), com duas casas decimais.

```
double yt = estabilizador * (1.0 - exp(-t / T));
```

Para gerar o arquivo `dadosA.csv`, um laço varia t de 0 a 10 s em passos de 0,1 s e calcula a Equação 2 em cada instante. A condição do laço usa `tempo_final + 1e-5` para compensar o erro de acumulação do passo em ponto flutuante e garantir que t = 10 s seja gravado.

4.3 FUNÇÃO TIPOB()

```
float Kdc = estabilizador/amplitude;
float Mp = (yp - estabilizador)/estabilizador;
float fatorAmortecimento = ((-1)*log(Mp))/sqrt(3.14159265 * 3.14159265 +
log(Mp) * log(Mp));
float frequenciaNatural = 3.14159265/(tp * sqrt(1 - (fatorAmortecimento *
fatorAmortecimento)));
```

As quatro linhas acima implementam, respectivamente, as Equações 1, 5, 7 e 8, com π escrito como constante numérica. O programa imprime Kdc, ζ, ω_n e a função de transferência da Equação 9, cujo numerador é Kdc·ω_n^2, e cujos coeficientes do denominador são $2\zeta\omega_n$ e ω_n^2.

```
double wd = frequenciaNatural * sqrt(1.0 - fatorAmortecimento *
fatorAmortecimento);
double phi = acos(fatorAmortecimento);
double yt = estabilizador * (1.0 - (exp(-fatorAmortecimento * frequenciaNatural
* t)
            / sqrt(1.0 - fatorAmortecimento * fatorAmortecimento)) * sin(wd * t
+ phi));
```

Para o arquivo `dadosB.csv`, o laço calcula a frequência amortecida, a fase φ e a resposta da Equação 10, com o mesmo horizonte e passo do Tipo A.

5 RESULTADOS DO CASO DE TESTE E DISCUSSÃO

5.1 CASO A: SISTEMA DE 1ª ORDEM

Entradas: A = 2, y(∞) = 10, t_1 = 2 s e y(t_1) = 6,32. Pela Equação 1, $K_{\mathrm{DC}} = \frac{10}{2} = 5$. Pela Equação 3:

$$T = \frac{-2}{\ln\left(1 - \frac{6{,}32}{10}\right)} \approx 2{,}0\ s$$

Logo, pela Equação 4, $G_1(s) = \frac{5}{2s+1}$. O valor T ≈ 2 s é coerente com o ponto lido: 6,32 corresponde a 63,2% de 10 em t = 2 s.

5.2 CASO B: SISTEMA DE 2ª ORDEM SUBAMORTECIDO

Entradas: A = 2, y(∞) = 20, y_p = 23,4 e t_p = 0,45 s. Pela Equação 1, K_{DC} = 20/2 = 10 e, pela Equação 5, $M_p = \frac{23{,}4 - 20}{20} = 0{,}17$. Pelas Equações 7 e 8:

$$\zeta = \frac{-ln(0{,}17)}{\sqrt{\pi^2 + ln^2(0{,}17)}} \approx 0{,}49$$

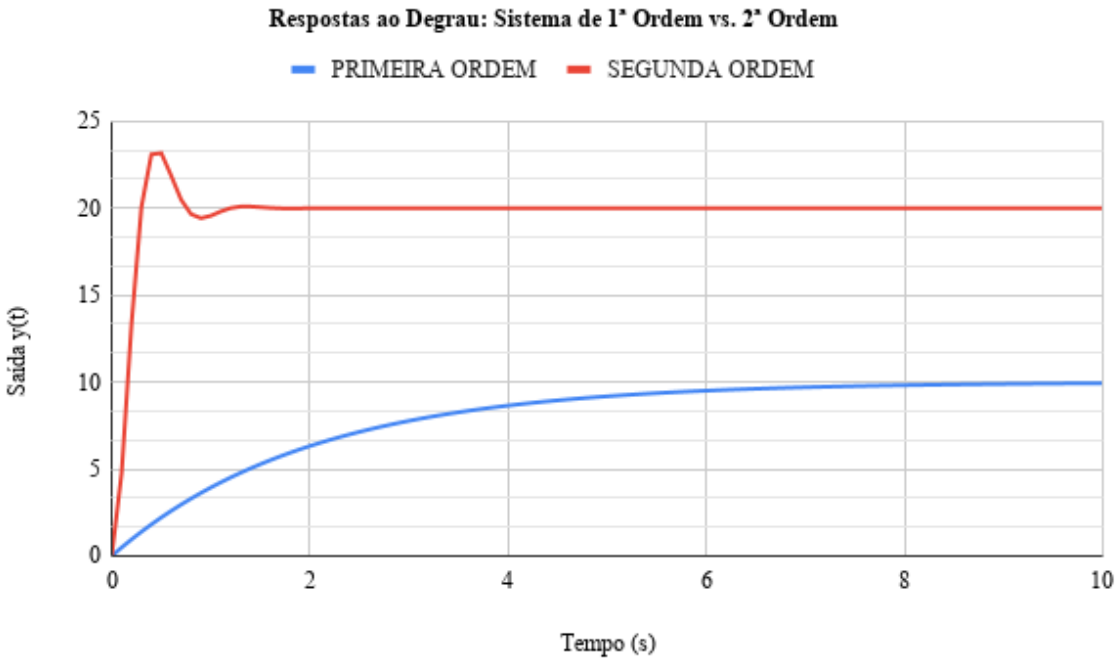
$$\omega_n = \frac{\pi}{0{,}45\sqrt{1 - 0{,}4913^2}} \approx 8{,}02\ rad/s$$

Pela Equação 9, $G_2(s) \approx \frac{642,4}{s^2 + 7,88s + 64,24}$. Arredondando ζ para 0,5, o resultado seria $\omega_n \approx 8,06$ rad/s e $G_2(s) \approx 650/(s^2 + 8,06s + 65,0)$, valores cerca de 1% diferentes dos acima.

5.3 ANÁLISE GRÁFICA

A Figura 1 apresenta as respostas dos dois sistemas, construídas a partir dos arquivos CSV exportados pelo programa.

Figura 1 – Respostas ao degrau de amplitude 2 dos sistemas de 1ª e 2ª ordem



Fonte: Elaborado pelo autor (2026).

Na Figura 1, observa-se que:

a) a curva do Tipo A sobe suavemente de 0 até 10, sem oscilar e sem ultrapassar o valor final; em t = 2 s vale 6,32 (63,2% de 10) e se acomoda em cerca de 8 s (4T), ou seja, estabiliza de forma assintótica, sem sobressinal;

b) a curva do Tipo B sobe rapidamente, ultrapassa 20 e atinge o primeiro pico de 23,4 em t_p = 0,45 s, o que corresponde ao sobressinal de 17%; depois oscila com amplitude decrescente e se acomoda em 20 em cerca de 1 s;

c) as linhas tracejadas marcam os valores de regime permanente, 10 e 20, e os círculos marcam os pontos usados na identificação.

Comparando os sistemas, o de 2ª ordem é mais rápido, mas apresenta resposta transiente oscilatória, porque é subamortecido e tem polos complexos conjugados. O de 1ª ordem é mais lento, porém não apresenta sobressinal. Como o passo do arquivo é 0,1 s e o

instante 0,45 s não é múltiplo dele, a maior amostra gravada no CSV do Tipo B é 23,16, em t = 0,5 s.

6 CONCLUSÃO

O programa em C identifica corretamente as funções de transferência dos dois sistemas a partir de poucas grandezas lidas no gráfico: a constante de tempo, no sistema de 1ª ordem, e o fator de amortecimento e a frequência natural, no de 2ª ordem. Para o caso de teste, obteve-se $G_1(s) = 5/(2s + 1)$ e $G_2(s) \approx 642{,}4/(s^2 + 7{,}88s + 64{,}24)$. O tratamento do separador decimal na exportação permite plotar os arquivos CSV diretamente em planilhas em português, e o gráfico evidencia a estabilização assintótica sem sobressinal do Tipo A e o transiente oscilatório do Tipo B.

O projeto está disponível no GitHub, no endereço: `https://github.com/marcow01/controle`. O repositório reúne o código-fonte em C, as versões compiladas para Windows e Linux, o exemplo com os dados do caso de teste proposto pelo professor, as planilhas CSV geradas pelo programa para cada tipo de sistema (dadosA.csv e dadosB.csv) e uma planilha em formato XLSX que mescla as duas, reunindo as respostas dos dois sistemas para a comparação gráfica.

APÊNDICE A – CÓDIGO COMPLETO

```
#include <stdio.h>
#include <math.h>

int tipoA(){
    float amplitude, estabilizador, tempo, ytempo;

    printf("1. Amplitude: "); scanf("%f", &amplitude);
    printf("2. Valor de estabilizacao (ex: 10): "); scanf("%f", &estabilizador);
    printf("3. Tempo t1: "); scanf("%f", &tempo);
    printf("4. Valor de y para tempo t1: "); scanf("%f", &ytempo);

    float Kdc = estabilizador/amplitude;
    double T = ((-1)*tempo)/log(1-(ytempo/estabilizador));

    printf("\nGanho Estatico (Kdc): %.4f\n", Kdc);
    printf("Constante de Tempo (T): %.4f s\n", T);
    printf("G(s) = %.2f / (%.2fs + 1)\n", Kdc, T);

    FILE *arquivo = fopen("dadosA.csv", "w");
    if (arquivo != NULL) {
        fprintf(arquivo, "Tempo (s);y(t)\n");
        double tempo_final = 10.0;
        double passo = 0.1;
        for (double t = 0; t <= tempo_final + 1e-5; t += passo) {
            double yt = estabilizador * (1.0 - exp(-t / T));
            char bufT[32], bufY[32];
            snprintf(bufT, sizeof(bufT), "%.2f", t);
            snprintf(bufY, sizeof(bufY), "%.4f", yt);
            for (char *p = bufT; *p; p++) if (*p == '.') *p = ',';
            for (char *p = bufY; *p; p++) if (*p == '.') *p = ',';
            fprintf(arquivo, "%s;%s\n", bufT, bufY);
        }
        fclose(arquivo);
        printf("\nArquivo 'dadosA.csv' gerado com sucesso!\n");
    }
    return 0;
}

int tipoB(){
    float amplitude, estabilizador, yp, tp;

    printf("1. Amplitude: "); scanf("%f", &amplitude);
    printf("2. Valor de estabilizacao (ex: 20): "); scanf("%f", &estabilizador);
    printf("3. Valor maximo do primeiro pico (yp, ex: 23.4): "); scanf("%f", &yp);
    printf("4. Tempo do primeiro pico (tp, ex: 0.45): "); scanf("%f", &tp);

    float Kdc = estabilizador/amplitude;
    float Mp = (yp - estabilizador)/estabilizador;
    float fatorAmortecimento = ((-1)*log(Mp))/sqrt(3.14159265 * 3.14159265 + log(Mp) *
   log(Mp));
    float frequenciaNatural = 3.14159265/(tp * sqrt(1 - (fatorAmortecimento *
   fatorAmortecimento)));

    printf("\nGanho Estatico (Kdc): %.4f\n", Kdc);
    printf("Fator de Amortecimento: %.4f\n", fatorAmortecimento);
    printf("Frequencia Natural: %.4f rad/s\n", frequenciaNatural);
    printf("G(s) = %.2f / (s^2 + %.2fs + %.2f)\n", Kdc * frequenciaNatural *
   frequenciaNatural, 2 * fatorAmortecimento * frequenciaNatural, frequenciaNatural *
   frequenciaNatural);

    FILE *arquivo = fopen("dadosB.csv", "w");
    if (arquivo != NULL) {
        fprintf(arquivo, "Tempo (s);y(t)\n");
        double wd = frequenciaNatural * sqrt(1.0 - fatorAmortecimento *
   fatorAmortecimento);
        double phi = acos(fatorAmortecimento);
        double tempo_final = 10.0;
        double passo = 0.1;
```

```
        for (double t = 0; t <= tempo_final + 1e-5; t += passo) {
            double yt = estabilizador * (1.0 - (exp(-fatorAmortecimento *
   frequenciaNatural * t) / sqrt(1.0 - fatorAmortecimento * fatorAmortecimento)) *
   sin(wd * t + phi));
            char bufT[32], bufY[32];
            snprintf(bufT, sizeof(bufT), "%.2f", t);
            snprintf(bufY, sizeof(bufY), "%.4f", yt);
            for (char *p = bufT; *p; p++) if (*p == '.') *p = ',';
            for (char *p = bufY; *p; p++) if (*p == '.') *p = ',';
            fprintf(arquivo, "%s;%s\n", bufT, bufY);
        }
        fclose(arquivo);
        printf("\nArquivo 'dadosB.csv' gerado com sucesso!\n");
    }
    return 0;
}

int main(){
    char tipo;
    printf("Escolha o tipo: (A) ou (B) \n");
    scanf(" %c", &tipo);

    if(tipo == 'A' || tipo == 'a'){
        tipoA();
    }
    else if(tipo == 'B' || tipo == 'b'){
        tipoB();
    }
    else{
        printf("Tipo invalido.\n");
    }
    return 0;
}
```